

SKILL WEEK-2

STEP 1: Open Ubuntu

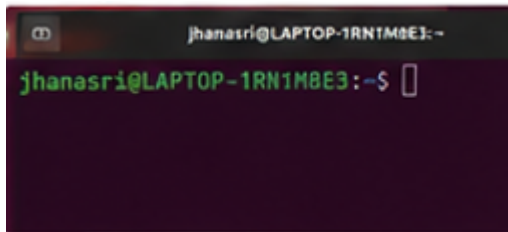
Open **Ubuntu** from the Start menu.

You should see:

```
jhanasri@LAPTOP-1RN1M8E3:~$
```

If you're somewhere else, go to your home folder:

```
cd ~
```



STEP 2: Create the OSSP folder

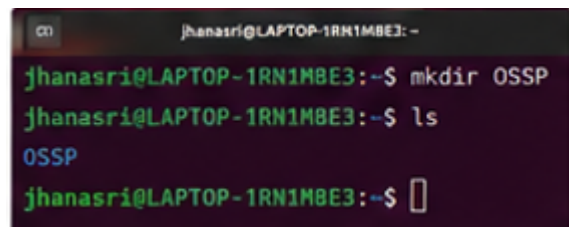
```
mkdir OSSP
```

Check:

```
ls
```

You should see

```
OSSP
```



STEP 3: Enter the folder

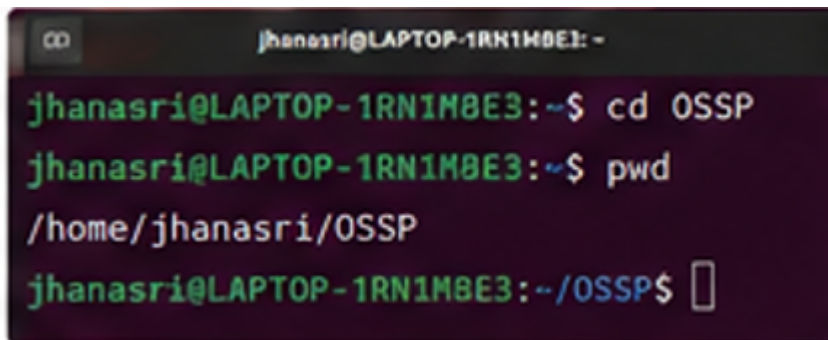
cd OSSP

Verify:

pwd

Output:

/home/jhanasri/OSSP

A terminal window with a dark background and green text. The prompt is 'jhanasri@LAPTOP-1RN1M8E3: ~'. The user enters 'cd OSSP' and the prompt changes to 'jhanasri@LAPTOP-1RN1M8E3: ~/OSSP'. Then the user enters 'pwd' and the output is '/home/jhanasri/OSSP'. The prompt then changes to 'jhanasri@LAPTOP-1RN1M8E3: ~/OSSP\$' with a cursor.

```
jhanasri@LAPTOP-1RN1M8E3: ~$ cd OSSP
jhanasri@LAPTOP-1RN1M8E3: ~/OSSP$ pwd
/home/jhanasri/OSSP
jhanasri@LAPTOP-1RN1M8E3: ~/OSSP$
```

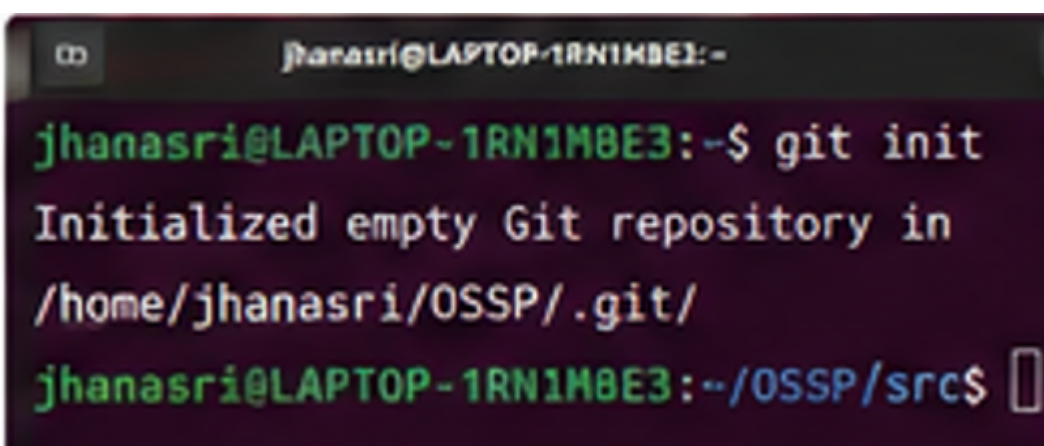
STEP 4: Initialize Git

git init

Expected output:

Initialized empty Git repository...

This creates the hidden `.git` directory automatically.

A terminal window with a dark background and green text. The prompt is 'jhanasri@LAPTOP-1RN1M8E3: ~'. The user enters 'git init' and the output is 'Initialized empty Git repository in /home/jhanasri/OSSP/.git/'. The prompt then changes to 'jhanasri@LAPTOP-1RN1M8E3: ~/OSSP/src\$' with a cursor.

```
jhanasri@LAPTOP-1RN1M8E3: ~$ git init
Initialized empty Git repository in
/home/jhanasri/OSSP/.git/
jhanasri@LAPTOP-1RN1M8E3: ~/OSSP/src$
```

STEP 5: Create folders

Run one command:

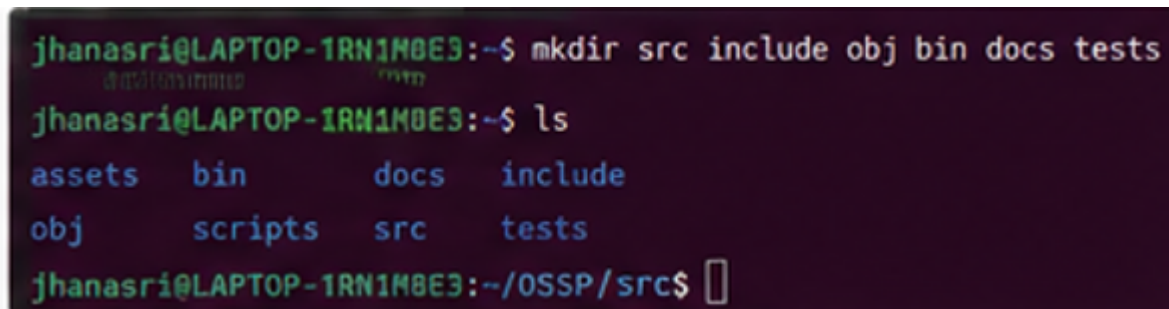
```
mkdir src include obj bin docs tests scripts assets
```

Check:

ls

You should see

assets
bin
docs
include
obj
scripts
src
Tests



```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir src include obj bin docs tests
jhanasri@LAPTOP-1RN1M8E3:~$ ls
assets  bin      docs    include
obj     scripts  src     tests
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/src$
```

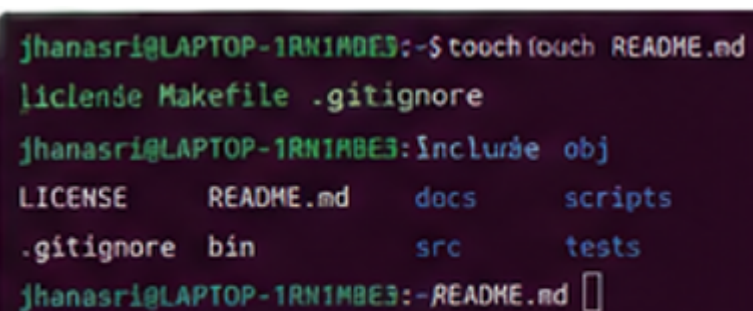
STEP 6: Create project files

Run:

```
touch README.md LICENSE Makefile .gitignore
```

Check:

Ls



```
jhanasri@LAPTOP-1RN1M8E3:~$ touch README.md
LICENSE Makefile .gitignore
jhanasri@LAPTOP-1RN1M8E3:include obj
LICENSE README.md docs scripts
.gitignore bin      src  tests
jhanasri@LAPTOP-1RN1M8E3:~README.md
```

STEP 7: Create source files

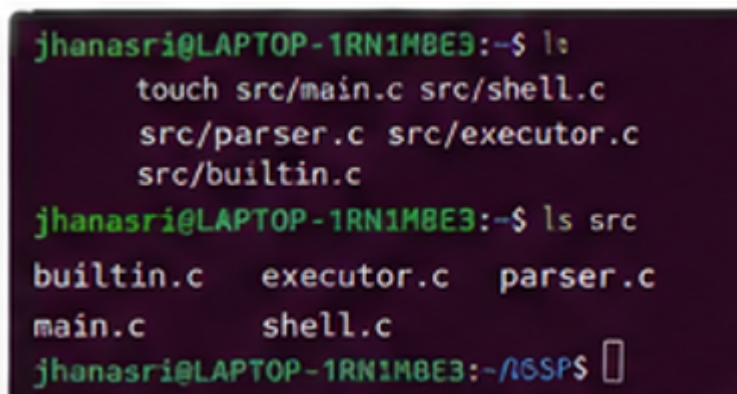
```
touch src/main.c
touch src/shell.c
touch src/parser.c
touch src/executor.c
touch src/builtin.c
```

Check:

ls src

Expected:

```
builtin.c
executor.c
main.c
parser.c
Shell.c
```

A terminal window with a dark background and green text. The user 'jhanasri' is at a laptop with ID '1RN1M8E3'. They run 'ls' and then 'touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c'. Then they run 'ls src' and see the output: 'builtin.c executor.c parser.c main.c shell.c'. The prompt changes to '~/#SP\$' with a cursor.

```
jhanasri@LAPTOP-1RN1M8E3:~$ ls
touch src/main.c src/shell.c
src/parser.c src/executor.c
src/builtin.c
jhanasri@LAPTOP-1RN1M8E3:~$ ls src
builtin.c  executor.c  parser.c
main.c    shell.c
jhanasri@LAPTOP-1RN1M8E3:~/#SP$
```

STEP 8: Create header files

```
touch include/shell.h
touch include/parser.h
touch include/executor.h
touch include/builtin.h
```

Check:

ls include

Expected:

builtin.h
executor.h
parser.h
Shell.h

```
jhanasri@LAPTOP-1RN1M8E3:~$ ls  
include/executor.h  
include/parser.h  
include/shell.h  
include/builtin.h  
jhanasri@LAPTOP-1RN1M8E3:~$ ls include  
builtin.h  executor.h  
parser.h  shell.h  
jhanasri@LAPTOP-1RN1M8E3:~/OSSPS$
```

STEP 9: Create documentation files

touch docs/design.md
touch docs/report.pdf

```
jhanasri@LAPTOP-1RN1M8E3:~$  
touch docs/design.md docs/report.pdf  
jhanasri@LAPTOP-1RN1M8E3:~$ ls docs  
design.md  report.pdf  
jhanasri@LAPTOP-1RN1M8E3:~/OSSPS/src$
```

STEP 10: Create test files

touch tests/test_parser.c
touch tests/test_executor.c

```
jhanasri@LAPTOP-1RN1M8E3:~$ ls
touch tests/test_parser.c tests/test_executor.c
jhanasri@LAPTOP-1RN1M8E3:~$ ls tests
ls tests
test_executor.c  test_parser.c
jhanasri@LAPTOP-1RN1M8E3:~/OS9/src$
```

STEP 11: Create helper script

touch scripts/[run.sh](#)

```
jhanasri@LAPTOP-1RN1M8E3:~$ touch touch README.md
LICENSE Makefile .gitignore
jhanasri@LAPTOP-1RN1M8E3:~$ include obj
LICENSE README.md docs scripts
.gitignore bin src tests
jhanasri@LAPTOP-1RN1M8E3:~$ cd README.md
```

STEP 12: Create image placeholder

touch assets/architecture.png

The document lists this image under [assets/](#); creating an empty placeholder file satisfies the required structure.

STEP 13: Check the complete structure

Run:

tree

If you get:

tree: command not found

Install it:

sudo apt install tree -y

Then run:

```
tree
```

You should see a structure similar to:

OSSP

```
|— assets
|   |— architecture.png
|— bin
|— docs
|   |— design.md
|   |— report.pdf
|— include
|   |— builtin.h
|   |— executor.h
|   |— parser.h
|   |— shell.h
|— obj
|— scripts
|   |— run.sh
|— src
|   |— builtin.c
|   |— executor.c
|   |— main.c
|   |— parser.c
|   |— shell.c
|— tests
|   |— test_executor.c
|   |— test_parser.c
|— .gitignore
|— LICENSE
|— Makefile
|— README.md
```

STEP 14: Verify Git

Run:

```
git status
```

You should see that Git has detected the new files.

Step 15: Open the **src** folder

```
cd ~/OSSP/src
```

Step 16: Create a new C file

```
touch fork_exec.c
```

Check:

```
ls
```

Step 17: Open the file

```
nano fork_exec.c
```

Step 18: Paste this code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    char command[100];
```

```
    printf("Enter Linux Command: ");
```



```
scanf("%s", command);

pid_t pid = fork();

if(pid < 0)
{
    printf("Fork Failed!\n");
    return 1;
}
else if(pid == 0)
{
    printf("Child Process\n");
    printf("Child PID = %d\n", getpid());

    execlp(command, command, NULL);

    printf("Command Not Found!\n");
}
else
{
    wait(NULL);

    printf("Parent Process\n");
    printf("Parent PID = %d\n", getpid());
}
```

```
    return 0;  
}
```

Step 19: Save the file

Press:

Ctrl + O

Press **Enter**

Then:

Ctrl + X

Step 20: Check whether GCC is installed

```
gcc --version
```

If you see a version number, continue.

If not, install it:

```
sudo apt install build-essential -y
```

Step 21: Compile the program

```
gcc fork_exec.c -o fork_exec
```

Step 22: Check the executable

ls

You should see:

fork_exec

Step 23: Run the program

./fork_exec

Step 24: Test with **ls**

Input:

ls

Step 25: Run again

./fork_exec

Input:

pwd

Step 26: Run again

./fork_exec

Input:

date

Step 27: Run again

./fork_exec

Input:

whoami

Step 28: View the process tree (optional but useful)

pstree

If `ps`tree is not installed:

sudo apt install psmisc -y

Then run:

pstree

Step 29: Trace system calls (if required)

Run:

strace ./fork_exec

If `strace` is not installed:

sudo apt install strace -y

Then run the command again.

Step 30: Check Git status

cd ~/OSSP

git status